Parallelizing the Geometric Brownian Motion Algorithm

ZE HONG WU, New Jersey Institute of Technology, United States of America

Geometric Brownian Motion (henceforward GBM) is a probability-based mathematical process in which a variable moves over time in a random motion. It appears most significantly in the context of financial prediction to simulate potential stock price movements. This report discusses efforts to apply parallel computing technologies and practices to the GBM algorithm and the resulting findings.

Additional Key Words and Phrases: Parallel Computing, Parallelization, Monte Carlo Simulation, Brownian Motion, Geometric Brownian Motion, OpenMP, CUDA, OpenMPI

ACM Reference Format:
Ze Hong Wu. 2018. Parallelizing the Geometric Brownian Motion Algorithm. 1, 1 (May 2018), 7 pages. https://doi.org/NOT-A-PUBLISHED-PAPER-DO-NOT-CLICK-THIS-LINK

1 Introduction

Monte Carlo simulations are a type of algorithm that uses large numbers of random samples to solve problems. Several examples of such simulations appear in coursework. The most notable example of this is the Pi estimator from the Module 5 assignment, which uses a dartboard approach of randomly generating points in a square and counting the ratio of points that fall within an enclosed circle.

Geometric Brownian Motion is a type of Monte Carlo simulation. It is based on the Brownian Motion concept, whereby a variable changes value across a number of steps through a "random walk". The "Geometric" prefix indicates that the random walk is moderated by an exponential. GBM is most well known for its use in the Black-Scholes model of financial modeling; the specifics of this model are beyond the scope of this report and are thus not discussed.

The GBM simulation revolves around the following equation:

$$S_t = S_0 * e^{X(t)}$$

... where

$$X(t) = (\mu + \frac{\sigma^2}{2})t + \sigma W(t)$$

... with the following variable definitions:

- t: A time delta from an arbitrarily selected reference point.
- S: A variable to be subjected to a GBM simulation, usually the price of a stock.
- S_0: The value of S at $t = 0$.
- S_t: The value of S at some time t where $t > 0$.

Author's Contact Information: Ze Hong Wu, zw4@njit.edu, New Jersey Institute of Technology, Newark, New Jersey, United States of America.

- μ: “Percentage drift”, representing an expected return rate over time. This value is constant throughout the basic GBM simulation; more advanced GBM simulations may have a variable drift.
- σ: “Percentage shock”, representing the variable’s likelihood to deviate from the expected drift. This value is constant.
- $W(t)$: "Wiener process", a Brownian Motion function that returns random values from the normal distribution $N(0, \sqrt{t})$.

In a single GBM simulation, the variable S takes a number of "steps" whose magnitude and direction are determined by the equation above. At the end of the simulation, the variable has effectively carried out a random march and ended at a potential final value. By running many GBM simulations, it is possible to generate a range of likely paths and end points for the variable. In the context of finance, GBM simulates how a stock’s price may change over a given time delta.

2 Parallelizability

Monte Carlo simulations satisfy the major design requirements for efficient parallelization: The algorithm workload can be easily and proportionally broken down into sub-workloads that can each be run independently of each other. Some have described Monte Carlo problems as "embarrassingly parallelizable"[1], given the ease of parallelizing the logic behind most Monte Carlo simulations.

GBM similarly satisfies the requirements for parallelization. Every GBM run is independent of every other run, allowing machines or processes to compute runs in parallel. Furthermore, steps in a GBM run are not dependent on immediately previous steps; it is possible to calculate the value of S for any number of time units after a reference point[2]. This means that it is possible to parallelize the steps in a GBM simulation as well; each step S_t can calculate its value using a common S_0 as a reference point.

3 Question and Hypothesis

The primary goal of this project is to determine the potential speedup levels that parallelization can potentially provide to the GBM process.

The primary question is: How do different parallel computing tools affect the speed of completion of the GBM algorithm for various problem sizes?

For the purpose of this project, I chose to evaluate the following three tools: OpenMP, Message Passing Interface (MPI), and NVIDIA CUDA.

4 Project Context

This project is not part of any ongoing research work and I do not anticipate actively using this work in a future project. However, the work done for this project may become relevant for future research. I am currently undergoing an interview process for an internship role involving GPU acceleration of highly compute-intensive algorithms, and the lessons learned from this project may be applicable to that role in the event of an offer.

5 Key Prior Work

During my research I discovered a similar project evaluating the possibility of parallelizing GBM, by Cannon Johnson, a fellow NJIT student. I elected to not closely review this work until after I have completed my algorithm tests, so as to avoid unintentionally contaminating my work with concepts and approaches taken from his.

Cannon's project placed focus on the effects of different hardware on the same algorithms, in the process focusing on two parallelization approaches. For my project I focused more on the effects of several parallelization approaches on the same hardware context, deprioritizing a potential discussion on how different CPUs/GPUs might affect performance across platforms.

During my research I did not discover any other benchmarks I could compare my work against. This was unfortunate as this means I do not have a point of reference to judge the competence of my parallel implementations against.

6 Methodology

For this project, my approach was as follows:

(1) Design the GBM algorithm in serial.
(2) For each of several potential parallelization solutions (OpenMP, MPI, OMP+MPI, CUDA), design a parallelized version of the serial GBM.
(3) Run strong and weak scaling tests against each version, using various problem sizes and parallel unit counts.
(4) Review the results and evaluate significance and takeaways.

For the strong scale tests, I set the problem size to N=2048 (number of GBM runs and number of steps in each run). For the weak scale tests, I used N $\in$ [32, 64, 128, 256, 512, 1024, 2048]. For both tests, I scaled the parallel computing units for each approach in the following manner: n $\in$ [1, 2, 4, 8, 16, 32, 64].

There are some exceptions to this general approach:

(1) NVIDIA GPUs divide threads into warps, each containing 32 threads. All threads in a warp get allocated at once, even if a workload requires less than the full 32 threads. As a result I elected to use 1 warp of 32 threads as the unit of parallel computing for the CUDA approach.
(2) Additionally, due to an initial design choice of not exceeding the 1024 thread limit in a GPU block, compounded by an issue with low Bridges2 GPU compute units, I removed N=32 and n=64 from the list of problem sizes and parallel compute units for the CUDA tests.
(3) For the MPI+OMP approach, I set a fixed OMP thread count of 32.

All testing for this project occurred on the Bridges2 HPC cluster of the Pittsburgh Supercomputing Center. See the Bridges2 documentation for a full discussion of its hardware.

7 Obstacles and Observations

During the design and testing process, I encountered a number of obstacles and events of interest.

7.1 Thread-safe random number generating

During the initial iteration of the different parallel code versions, I used C's `rand()` function as part of the procedure to generate numbers in a normal distribution. However, since the `rand()` function is not thread-safe, this led to a noticeable lack of speedup in my parallel approaches. `rand` uses an internal counter that is common to all threads. With many threads competing for access, the bottleneck slows down computing. This, combined with another issue discussed below, resulted in parallel code running no faster than serial. I resolved this by using the `rand_r()` function instead, which is guaranteed to be thread-safe.

7.2 Static variables and race conditions

During the initial iteration of the different code versions, I used a function that generated random numbers from a normal distribution by transforming values from `rand_r()` (which returns values from a uniform distribution). This function made use of static variables, which by default introduces race conditions as all threads access the same static variable in memory. I resolved this through by a combination of switching to another approach that eliminated most of the static variables and applying the `_Thread_local` keyword to the last static variable that I could not eliminate due to an inherent requirement with `rand_r()`. The keyword ensures that each thread has a local copy, eliminating any potential bottlenecks.

8 Codebase

The full code of the various GBM approaches can be found in the following GitHub repository. Other sections of this report may refer to function calls or keywords used in parts of the code.

9 Observations

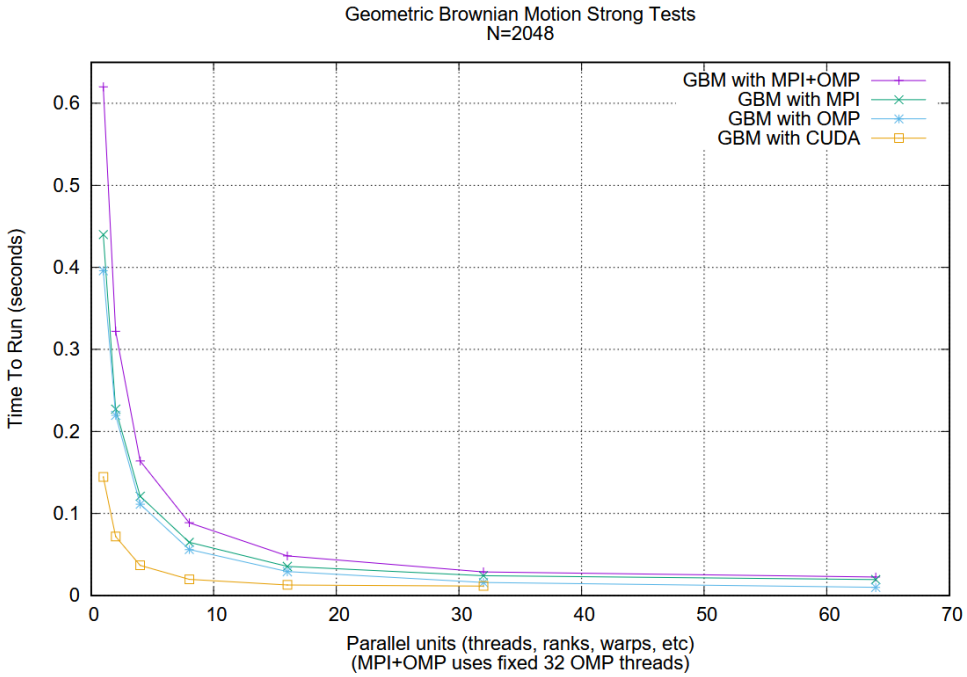

Fig. 1. Chart of strong scale test results, generated using gnuplot. The problem size is a fixed 2048 GBM runs each with 2048 steps.

Figure 1 shows the results of the strong tests against the several approaches, while Figure 2 on page 5 shows a zoomed-in version of Figure 1 that reveals the detailed interactions near the bottom of the graph.

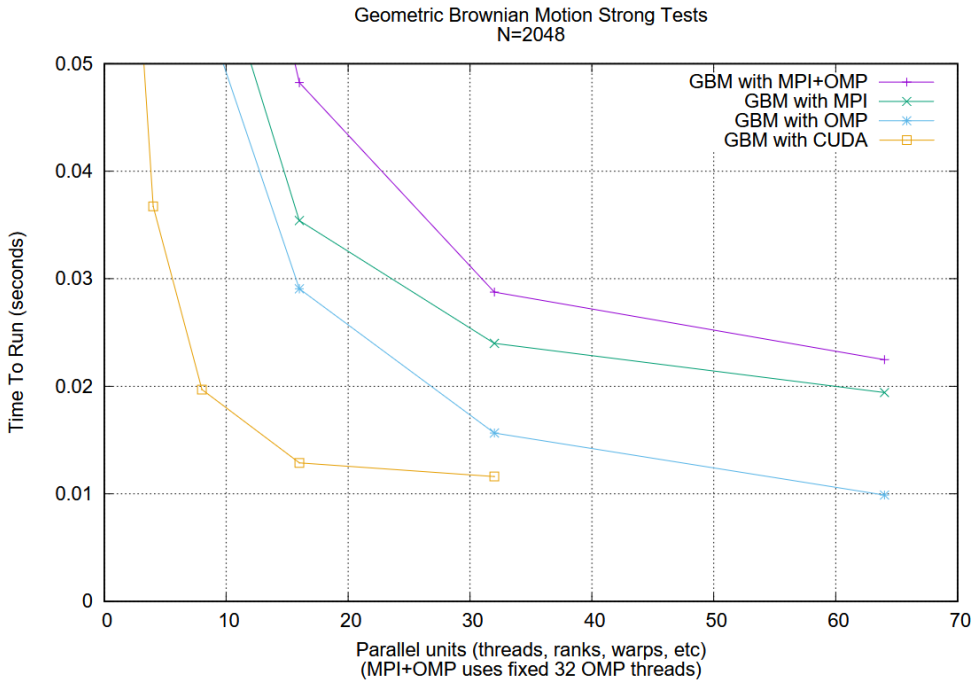

Fig. 2. A zooming in of Fig. 1 to reveal details near the bottom of the graph.

All 4 approaches showed noticeable improvements as the number of parallel units increased. The CUDA approach yielded faster runtimes for smaller parallel units compared to the other approaches. However, a review of Figure 2 shows its lead dropping off at higher unit counts.

The combined MPI+OMP approach was the slowest of the four. This is likely due to the combined overhead of setting up MPI instances and OMP threads, combined with an `MPI_Gather()` call to collect partial GBM runs into a full array of runs, which introduced time costs.

The MPI alone approach was slower than the OMP alone approach. This may be caused by the inherently time-costly nature of inter-process communication. The `MPI_Gather()` call required the various MPI processes to send the total GBM run data, which at N=2048 includes 4194304 double-precision floating point numbers, to process rank 0. OMP threads in my implementation, meanwhile, do not majorly communicate with each other; they work on different sections of a contiguous memory block and the final data is already assembled by the time the last OMP thread concludes.

Figure 3 (see Page 6) shows the results of weak tests against the several approaches. See the Methodology section for a detailed discussion of the problem size and parallel computing unit combinations.

While CUDA provides strong performance at lower values of N, the OMP approach eventually resulted in faster runtimes at N=2048.

Using the N=2048 test runs, I calculated the following speedup values for each of the approaches:

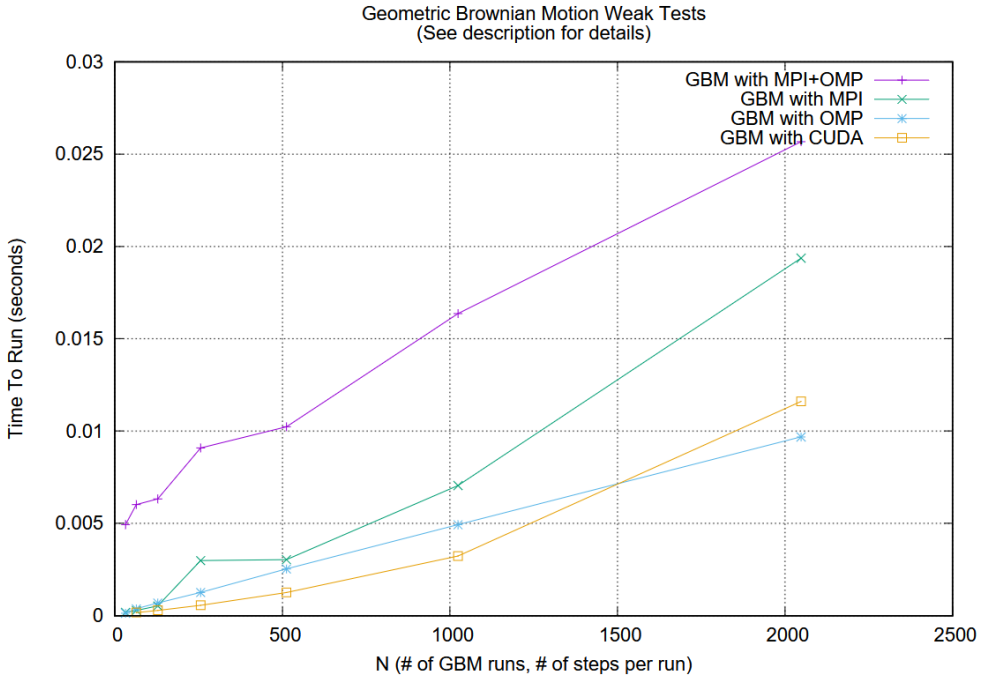

Fig. 3. Chart of strong scale test results, generated using gnuplot. N ∈ [32, 64, 128, 256, 512, 1024, 2048].

Approach	Time To Run	Speedup	Efficiency
Serial	0.329573	1.0	1.0
MPI+OMP	0.022481	14.66	0.1527
MPI	0.019368	17.02	0.2659
OMP	0.009679	34.05	0.532
CUDA	0.011607	28.39	0.0277

Efficiency for the CUDA approach is measured using 1 thread as a single processor.

CUDA's efficiency looks quite appalling from this perspective, when compared to the other approaches. However, we must keep in mind that different approaches have different parallelization limits. 1024 CUDA threads fit within 1 GPU block, of which many blocks exist in a GPU. Meanwhile, for reference, the Bridges2 standard computing node has 128 combined physical CPU cores. The resource ceiling in this context is much lower proportion-wise for MPI processes and OMP threads.

10 Reflection and Further Work

Due to computing resource limits, there were some initial design choices (such as those involving the CUDA tests and thread counts) that I could not revise on without depleting my compute units. This could have been resolved by carefully thinking through my initial design choices and identifying the issues beforehand instead of realizing it after-the-fact.

During my research I found two data points that seemed to contradict each other. Many of the academic and scholarly resources I found discussing the math behind GBM described its function as returning valid predicted values for any time delta `t+h` given `t` as the reference point and `h` as any time difference. However, all of the GBM implementations (primarily written in Python) that I found calculated intermediate GBM values using the immediately previous value, in effect finding the intermediate values serially. I have not fully figured out the significance of this discrepancy; depending on which finding is more legitimate, the core logic behind each of my implementations may need to be adjusted to account for the potential serial nature of the inner loop calculating intermediate values for a single GBM run.

This project leaves open potential avenues that can be further investigated. A non-exhaustive list is below.

(1) How does the performance for the different approaches scale for larger values of N (4096, 9182, etc)?
(2) How does this parallelism performance generalize to a workflow incorporating GBM and other algorithms, such as a full Black-Scholes market simulation model?
(3) How might an OpenCL implementation compare to the approaches tested here? (I elected to not attempt an OpenCL implementation due to its complexity and due to needs to balance time/effort expenditures with other efforts.)

Acknowledgments

This work used Bridges-2 at Pittsburgh Supercomputing Center through allocation cis240001p from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

References

[1] Jeffrey S. Rosenthal. 2000. Parallel computing and Monte Carlo algorithms. *Far East Journal of Theoretical Statistics* 4 (2000), 27 pages. https://probability.ca/jeff/ftpdir/para.pdf

[2] Karl Sigman. 2006. *Geometric Brownian motion.* Retrieved May 10, 2026 from https://www.columbia.edu/~ks20/FE-Notes/4700-07-Notes-GBM.pdf